



EEDOS: an energy-efficient and delay-aware offloading scheme based on device to device collaboration in mobile edge computing

Ramtin Ranji¹ · Ali Mohammed Mansoor¹ · Asmiza Abdul Sani¹

Published online: 14 August 2019
© Springer Science+Business Media, LLC, part of Springer Nature 2019

Abstract

Device to device (D2D) communication and mobile edge computing (MEC) are two promising technologies in fifth generation (5G) cellular mobile communication. Besides MEC, a new task offloading technique attracts the attention as D2D collaboration. However, there is lack of integrated D2D and MEC framework to address the energy and delay costs in a joint approach. This work, proposes an energy efficient and delay-aware offloading scheme (EEDOS) based on D2D collaboration in MEC. In EEDOS, mobile devices can offload their task to the MEC or an idle mobile device in their proximity. The task execution and offloading to the MEC or an idle nearby device is formulated, and the optimization problem is defined. The whole process of allocating proper offloading destination is designed in the edge server. EEDOS, classifies offloading requests according to the deadline and energy constraint of requesting device. Then, it finds the proper offloading destination by utilising the maximum matching with minimum cost graph algorithm. Through simulation, we show that EEDOS achieves 95 percent of energy efficiency in comparison of no-offloading task execution and outperforms existing studies in term of energy efficiency with an improved delay in task execution. Moreover, EEDOS is capable of performing more successful task offloading and requires less edge server resources.

Keywords Task offloading · Device to device communication · Mobile edge computing · Energy efficiency

1 Introduction

In the last decade, mobile phones transformed from a simple device for making phone calls and a multimedia gadget, to a fully functional computing device. Today, using smartphones have become part of every day's activity, and the users need mobile phones to operate for extended hours throughout the day. However, their portability and design constraint, made them less capable for today's resource-hungry applications. Augmented reality, face recognition and online gaming are few examples of applications that need high computational capability and low latency network [1]. Improving the pro-

cessing power of mobile devices to support more features, without a significant evolution in the design of their batteries is responsible for the fast battery drainage, and make them incapable of long hours operating [2].

Internet of Things (IoT) devices are deployed widely and the number of devices are expected to grow exponentially in the coming years [3]. IoT devices are extremely resource limited in terms of processing power, storage, and battery capacity. However, in some applications, they need a low latency response like gas leakage scenario [4]. As another example, Vehicles to Everything (V2X) cannot perform well without a real-time response [5]. Therefore, low delay operation of mobile/IoT devices is an important challenge for hardware and network engineers.

Cloud computing is one of the technologies that can help to save the energy of mobile and IoT devices and it is deployed broadly. In cloud computing, devices can offload their computational-intensive tasks to the cloud server, where the immense volume of computing power exists to facilitate the work of resource-limited devices [6,7]. However, the imposed delay by cloud computing is not suitable for many of today's delay sensitive applications [8]. Devices can commu-

✉ Ali Mohammed Mansoor
ali.mansoor@um.edu.my

Ramtin Ranji
ramtinranji@gmail.com

Asmiza Abdul Sani
asmiza@um.edu.my

¹ Department of Software Engineering, Faculty of Computer Science and IT, University Malaya, 50603 Kuala Lumpur, Malaysia

nicate with cloud servers through WiFi or broadband wireless technologies like Long Term Evolution (LTE). Fifth generation (5G) of cellular mobile network is the newest standard, based on the IEEE802.11ac standard of cellular technology. In the 5G design, energy consumption of the network is expected to decrease by the factor of 10 [9]. Mobile Edge Computing (MEC) and Device to Device (D2D) communication are two technologies in the 5G to save the energy of mobile/IoT devices and decrease their communication and computing delay.

MEC is proposed to overcome the long delay of task offloading in cloud computing. Moreover, MEC architecture avoids the time consumption for data transmission through the core network. MEC collects data from the end user or IoT devices and processes it at the edge of the network without sending it to the traditional cloud [10]. Offloading the computational-intensive tasks to the MEC will lead to saving the energy of those devices with a low delay. Also, real-time applications can be supported by the task offloading.

D2D communication was first introduced in the LTE Rel-12 standard of cellular network as a Proximity Service (ProSe) [11]. It is a promising technique in 5G to provide service for a resource-limited device by utilizing communication resources of a more powerful nearby device without the intervention of base station (BS) [12]. Devices close to the BS have high signal-to-noise ratio (SNR), as a result, they can be allocated high data rates, but devices that are far from the BS will have a low modulation degree and low data rate [13]. To alleviate this, devices in the proximity can enjoy the high D2D SNR value to communicate with a high data rate, despite their distance from BS. Devices in the 5G network are heterogeneous and have different computation and storage capabilities. As the number of devices connecting to a single cell increases in the 5G, there is a huge probability that, many powerful devices become available in the proximity of a resource-limited device. As a result, D2D collaboration is able to fully exploit task offloading. In this technique, devices with limited resources like CPU and energy, can offload their task to a more powerful device to save their energy and decrease the computation delay [14]. It must be noted that, this paper, put the attention mostly on the benefits of task execution out of the local hardware of IoT/Mobile devices.

This paper, propose an energy-efficient and delay-aware offloading scheme for MEC which is called EEDOS. In EEDOS, devices with low computation power or insufficient energy can offload their task to a nearby device or to the MEC. If MEC becomes overloaded, it sends the tasks of energy-limited devices to the cloud. In the next section, we review existing methods for computation offloading in the 5G network. In the current literature, there is a lack of comprehensive study that jointly considers the computational capability of MEC and idle devices as offloading

destinations, and considering the required energy as well as the execution time. Therefore, the contributions of this paper are (1) The offloading problem is formulated in detail with consideration of D2D and cellular communications as well as computation capability of MEC and idle devices in the network. (2) Proposing the scheme and offloading algorithms to reduce the energy consumption of devices and decrease the task delay. This is done by relaxing the offloading problem into two sub-problem considering the deadline of the tasks and then energy consumption of devices. Each of the sub-problems are solved with the polynomial complexity. Both users that suffer from insufficient energy and those who cannot meet the deadline of the task due to the lack of sufficient computational power can use EEDOS to offload their tasks. We explain our model and task offloading mechanism in Sect. 3. An extensive simulation is presented in Sect. 4 to evaluate the performances of EEDOS in LTE realistic channel. In order to verify the effectiveness of EEDOS in comparison with current studies, two of the most recent works on MEC offloading and D2D collaboration were simulated as well. The simulation results show that EEDOS is capable to achieve a steady energy efficiency without edge overload. Moreover, EEDOS outperforms existing studies in term of energy efficiency, delay, required edge CPU and number of successful task offloading. We highlight the research gap and present the future direction to improve the proposed framework in Sect. 5 and conclude the paper in Sect. 6.

2 Related work

Using MEC as an offloading destination is studied in [15,16]. In [15], authors consider the cost of task execution and input transmission to formulate energy optimization problem for energy efficient task offloading. Devices are classified based on the comparison of local execution and transmission cost with their current state. Devices that cannot meet the delay constraints and have limited energy are given the highest priority in the radio resource allocation process. Authors in [16] propose an integration of cloud, MEC and IoT to provide energy efficient task offloading framework under the latency requirement. Indeed, this work overcome the scalability problem of the considerable number of IoT offloading request by a lightweight request and admission framework. However, there is a lack in managing the nearby devices in the network to overcome the degradation of the latency in their framework. In [17] authors jointly addressed task offloading and resource allocation problem in MEC servers to maximize the benefits of users when they offload their tasks. They divide the problem in two task offloading and resource allocation sub-problems to reduce the complexity and solve it in polynomial time. The shared limitation of

these studies is neglecting the possibility of using the computational resources of nearby devices for task offloading.

Integrating the capabilities of MEC with the D2D collaboration is proposed by [18] for task offloading. This proposed framework shares computation resources of mobile devices to achieve minimum network-wide energy consumption for task executions. The authors proposed a framework which becomes a basis of the D2D crowd framework for 5G MEC in [19,20]. In these works, the authors propose computation capability, cellular link and D2D link models of device resources for task of mobile devices and task executions. The task execution model is presented in a way that a mobile device can perform the task locally or offload to a nearby device. The mechanism built a nearby graph and weight it with the energy cost of either local execution or offloading to the neighbor. The graph is resolved by the edge computing server that can notifies each device to run the task locally or offload to an effective neighbor. In [24], authors proposed a joint task assignment and resource allocation offloading system in MEC environment in which devices can use nearby devices as the helpers. However, the computational capability of MEC server is neglected in their study. Enabling low delay applications like augmented reality is an essential for developing 5G networks, but in [18,19], only the energy consumption of the device is taken into account which is not satisfactory for delay sensitive tasks. Moreover, the computation capability of MEC is excluded, while the proposed work in this paper employs it to run tasks offloading in order to save the energy of mobile devices and provide low latency.

D2D communication can increase the bit rate due to the lower distance between users and can decrease the energy consumption by using lower transmission power. However, in order to gain energy efficiency in D2D communication, several issues must be addressed first. In [21], authors specifies these issues as mode selection, user admission control and resource allocation. A Mesh Adaptive Direct Search Algorithm have been proposed to achieve energy efficiency in their study. Interested readers may refer to [22] for a comprehensive survey on the other studies concerning the aforementioned issues. In [23] authors consider computing resource, power, and channel allocation to exploit the benefits of D2D collaboration in MEC. Communication channel stability is another concern that authors in [25] proposed a dynamic energy-efficient data offloading scheduling algorithm to address it. They proposed a heuristic algorithm to decrease the energy consumption while considering the reliability of tasks.

In this paper, the energy efficient task offloading framework is presented by integrating the capabilities of nearby devices and edge computing servers. To the best of our knowledge, EEDOS is the first integration of D2D-enabled network, MEC and cloud computing technologies with a

delay awareness to provide an effective energy efficiency for mobile and IoT devices to meet the latency requirements.

3 Energy-efficient and delay-aware offloading scheme (EEDOS)

3.1 System architecture

The proposed scheme, EEDOS, is capable to rectify if a device is not able to meet the deadline of such sensitive tasks. This is achieved via offload the task to a nearest effective device or edge server. If a device suffers from the lack of sufficient energy, it can offload its task to three alternative options: a *neighbor device*, *edge server* or *cloud server*. The decision about either option can be made efficiently by MEC server. Figure 1 illustrates the network topology that integrates D2D-enabled user layer with MEC and Cloud computing layers.

3.2 Problem formulation

To define the optimization problem, we first describe the time and energy model for task execution and offloading to the different layers of the EEDOS. Table 1 represents the definition of the formulas and algorithms notations.

3.2.1 User/IoT device model

In EEDOS, user/IoT devices with the capability of D2D communication are in the first layer of the architecture. The computation capacity and communication details of the devices have been formulated. In the first stage of communication, devices can only communicate with the MEC layer. If MEC server decides that, D2D collaboration is the most efficient task execution, mobile devices can communicate directly without the intervention of the BS. The background services and tasks that the device cannot offload them, occupy the CPU of the device by θ . As a result, the current computation capacity of the device i is:

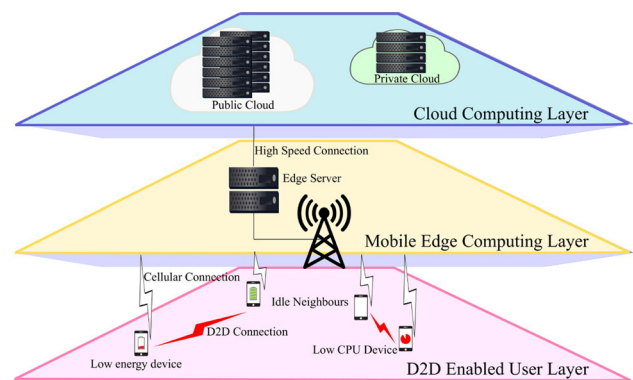


Fig. 1 Network topology of EEDOS

Table 1 Nomenclature used in this paper

Notation	Description
N	Set of devices were connected to MEC server
$\bar{N}$	Set of devices that send offloading request to MEC
f_i^0	CPU clock of the device $i \in N$
f_i^l	Current computation capacity of i
f_j^{d2d}	Allocated resources of j to perform task i
f_{mec}^0	Initial computation capacity of MEC server
f_{mec}	Current computation capacity of MEC server
θ	Current CPU load of i $0 < \theta \leq 1$
$\mu_i \in \{0, 1\}$	Denotes whether the task of i is offloaded to MEC
$\mu_{it} \in \{0, 1\}$	Whether the task i is delay-sensitive or not
R_i^{ul}, R_i^{dl}	Uplink, downlink data rate of i
P_i^{ul}, P_i^{dl}	Transmission, receive power of i
R_{ij}^{d2d}	Data rate of communication from device i to j
P_i^d	D2D transmission power of device i
ρ_i	Energy cost per CPU cycle for device i
I_i, O_i, Z_i	Input size, output and required CPU cycles of task i
B_i^{ul}, B_i^{dl}	Uplink and downlink cellular traffic for task i
T_i^{req}	Deadline for task i
T_i^l	Total time of performing task i locally
T_i^{mec}	Total time of offloading task i to MEC server
T_{ij}^{d2d}	Total time of offloading task of i to j
E_i^l	Required energy for device i for local execution
E_i^{mec}, E_i^{cc}	Required energy (device i) for MEC/cloud offloading
E_{ij}^{d2d}	Required energy for device i to offload the task to j
W_i^t	Current energy level of device i
W_i^{tr}	Threshold for device i to enter energy sensitive mode
G^e	Energy-efficient graph
G^t	Delay-aware graph
$E(a, b)$	Edge weight in graph from device a to b
S^t	Set of selected nodes in the delay-aware graph
S^e	Set of selected nodes in the energy-efficient graph

$$f_i^l = (1 - \theta)f_i^0 \quad (1)$$

Devices continuously provide feedback to the base station. This mechanism utilizes the transmission power of the device, current location, the noise of the channel. Each device has its unique SINR value at a given time. Each device in the network periodically reports its status to the base station as Channel Quality Indicator (CQI). In the interest of energy-efficiency to all users that report CQI feedback to the base station to each resource blocks, authors in [26] propose a partial CQI feedback to save the energy.

The required time for cellular traffic obtains as follow:

$$T_i^{ul} = B_i^{ul} / R_i^{ul} \quad (2)$$

$$T_i^{dl} = B_i^{dl} / R_i^{dl} \quad (3)$$

In EEDOS architecture, D2D SINR values are reported to the base station. According to the following Shannon Formula, MEC can calculate the data rate of the D2D communication from device i to j as R_{ij}^{d2d} .

$$C = R_{ij}^{d2d} = B \log_2 \left(1 + \frac{S}{N} \right) \quad (4)$$

Where C is for maximum capacity, B is for bandwidth, S is the signal power and N is the noise power.

3.2.2 MEC server computing capacity model

MEC layer is located near to the user and IoT devices. MEC servers is capable to communicate with the user and IoT devices directly on one side and cloud computing layer on the other side. MEC layer brings the capabilities of the cloud computing within the Radio Access Network (RAN). As a result, mobile/IoT devices that require real-time response can use the services with low latency. EEDOS considers the traditional cloud computing layer to integrate powerful resource-rich servers that are located in this layer to run complex tasks. There are different Virtual Machines (VM) in any MEC server to support different services simultaneously. Network manager can adjust the computation capacity of each VM statically or allocate it dynamically. MEC server specify the minimum computation capacity of CPU to provide scalability. The remaining computation capacity of the VM is obtainable through Formula 5 where μ_i has a binary value that indicates whether a task is offloaded to the MEC or not.

$$f_{mec} = f_{mec}^0 - \sum_{i=1}^n \mu_i f_i^{mec} \quad (5)$$

3.2.3 Task execution model

We denote the task by the following tuple:

$$(I_i, O_i, Z_i, B_i^{ul}, B_i^{dl}, T_i^{req}) \quad (6)$$

In EEDOS task model, I_i is the input size, O_i is the output size, Z_i is the required CPU cycles, B_i^{ul} and B_i^{dl} are the required cellular traffic, and T_i^{req} is the delay constraints (deadline) of the task. In the local execution procedure, if a device runs the task by its CPU, the total time of execution is the cumulative time of processing and cellular transfer as follows:

$$T_i^l = Z_i / f_i^l + T_i^{ul} + T_i^{dl} \quad (7)$$

According to the previous equation, the required energy to perform the task obtains from the Formula 8. On the other

hand, if $T_i^l > T_i^{req}$, device cannot meet the deadline so it can send the request for offloading decision to the MEC server. Similarly, if the current energy of the device is below the predefined threshold ($W_i^l \leq W_i^{tr}$) it can send the offloading request to the edge server.

$$E_i^l = Z_i \rho_i + P_i^{ul} T_i^{ul} + P_i^{dl} T_i^{dl} \quad (8)$$

If a device sends the offloading execution request, the edge server can decide between three options based on the following calculations:

If device i needs to offload its task to device j , it takes time to transfer the input of the task and receive the output. In addition, the overhead for j to perform the task must be considered. The total time and required energy for D2D offloading is obtainable as follows:

$$T_{ij}^{d2d} = I_i / R_{ij} + O_i / R_{ji} + Z_i / f_j^{d2d} + B_i^{ul} / R_j^{ul} + B_i^{dl} / R_j^{dl} \quad (9)$$

$$E_{ij}^{d2d} = P_i^d (I_i / R_{ij}) + P_j^d (O_i / R_{ji}) + \rho_j (Z_i / f_j^{d2d}) + P^u l_j (B_i^{ul} / R_j^{ul}) + P^d l_j (B_i^{dl} / R_j^{dl}) \quad (10)$$

If $T_{ij}^{d2d} \leq T_i^{req}$ device j will be a candidate for D2D offloading. On the other hand, MEC server can assign its resources to delay sensitive tasks with higher priority to save its resources to utilize more users. As a result, MEC server adjusts resource allocation to meet the delay required by the task as in Formula 11. Therefore, the required time for MEC offloading is calculated as Formula 12 and the required energy for a device to offloads the task is calculated as 13.

$$f_i^{mec} = Z_i / (T_i^{req} - T_i^{ul} - T_i^{dl}) \quad (11)$$

$$T_i^{req} \leq T_i^{mec} = Z_i / f_i^{mec} + T_i^{ul} + T_i^{dl} \quad (12)$$

$$E_i^{mec} = P_i^{ul} T_i^{ul} + P_i^{dl} T_i^{dl} \quad (13)$$

If edge server cannot accept the task and offloading the task to a proximate device is not effective for the requested device, the task is sent immediately to the cloud. Accordingly, EEDOS considers the energy that is required by the device i to offload its task to the MEC server. Due to the fact that, devices can only communicate with servers in the MEC layer, the cost of cloud offloading for mobile devices is calculated as follows: $E_i^{mec} = E_i^{cc}$.

The aim of this work is to minimize the energy consumption of energy-constrained devices for task execution. Meanwhile, for delay sensitive tasks, it is required to minimize the execution time considering the task deadline as follows:

$$\min \sum_{i=1}^n \left(\mu_{it} \left(\mu_i T_i^{mec} + \sum_{i \neq j, \mu_i=0} T_{ij}^{D2D} \right) + (1 - \mu_{it}) \left(\mu_i E_i^{mec} + \sum_{i \neq j, \mu_i=0} E_{ij}^{d2d} \right) \right) \quad (14)$$

3.3 Offloading selection algorithms

for the sake of optimization, EEDOS divides the algorithms of finding the offloading destinations into two stages. The first stage is for delay sensitive tasks that is required to assign the higher priority with deadline satisfaction. The second stage is for energy requirements which devices suffer from the lack of energy, advised to offload their task. To avoid the transmission overhead and delay in D2D crowd system [20] for the local execution that is imposed by MEC, EDDOS is capable to allow the device to make this decision locally. This can be done based on the following conditions:

$$W_i^t - E_i^l > W_i^{tr}, T_i^l \leq T_i^{req} \quad (15)$$

It means that the device has sufficient energy to perform the task and it can meet the delay requirement. In any other situation, the device will send an offloading request to the MEC server.

Figure 2a demonstrates the workflow of EEDOS scheme to find a proper destination for users' offload requests in the MEC server. As shown in the figure, devices with low energy and devices that have a delay-sensitive tasks send their offloading requests to the MEC. Then, according to the MEC offloading decision algorithm for delay sensitive devices (Algorithm 1), the MEC classifies requests based on their requirements (energy or delay), and then it classifies the nodes into three categories.

- Devices that have a task to offload
- Idle devices
- Allocated resources in VM to a particular device

Now, the MEC is capable to build the adjacency graphs as shown in Fig. 2b. The first graph builds for delay sensitive tasks (Delay-aware graph) to assign the resources of the MEC effectively to meet the deadline. After the graph construction, it is solved by maximum matching with minimum cost (Edmond's Blossom [27]) method. According to the result, MEC either selects and reserves its own resources or select a nearby device for D2D collaboration. Then, MEC reports the result to the requested devices and now they are able to offload their task via D2D communication or send it to the MEC.

In the second stage, MEC considers energy sensitive tasks. Consequentially, another weighted graph has been built upon

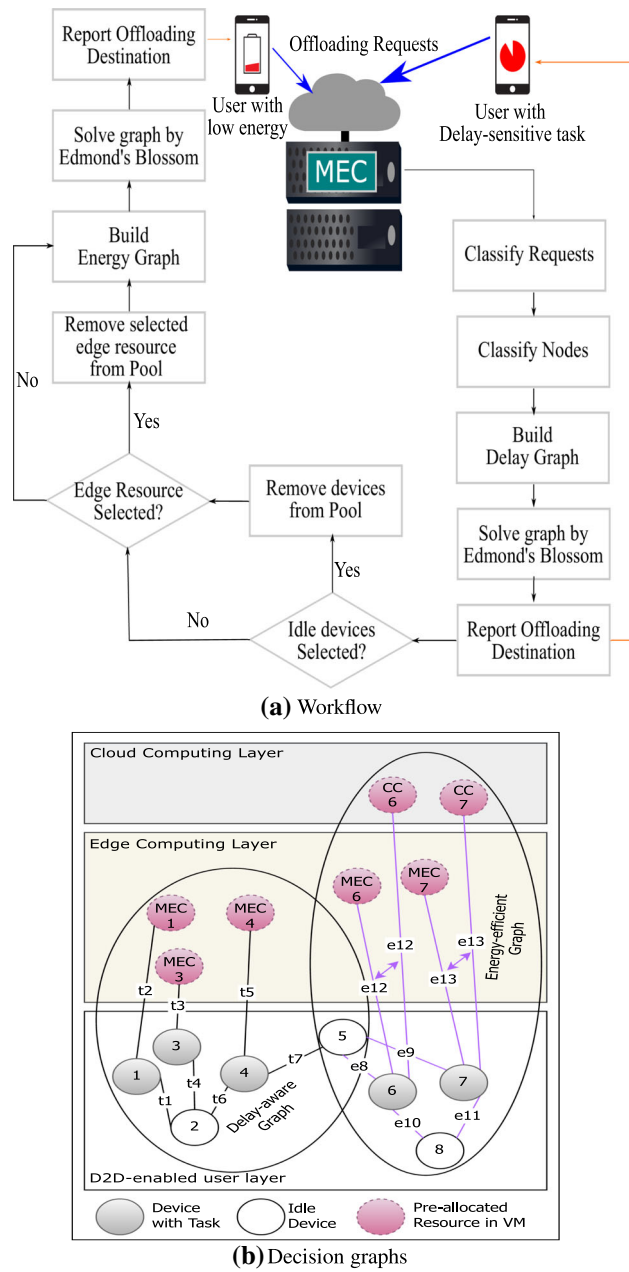


Fig. 2 EEDOS Scheme

the ability of D2D communication and the remaining capacity of MEC. Based on the result of the previous algorithm, it must omit some of the idle nodes in the proximity of the requested users because they are already selected to perform delay-sensitive tasks. Then, the energy-efficient graph is created and solved by the use of MEC offloading decision algorithm for energy sensitive devices (Algorithm 2). Similarly, MEC finds the maximum matching with minimum cost and inform all of the requested devices to offload their task to the selected neighbor or the MEC.

Figure 2b, illustrates the EEDOS topology and a sample scenario. Devices with task and idle devices are in D2D-enabled user layer, the VM which is dedicated to EEDOS in Edge server reserves its resources in Edge computing layer. If MEC becomes out-of-resource, it can forward the task to the cloud. Therefore, it must allocate resources in the cloud computing layer to prevent overloading. In the example scenario, suppose device 2 has been selected as the offloading destination of device 1, and devices 3 and 4 offload their tasks to the MEC. Therefore, device 5 has not been selected in the first stage for offloading. Therefore, MEC includes it in the energy-efficient graph again. In this graph, sufficient computational power is remained for MEC, the edge between MEC and requested devices will be created (like e12 edge between device 6 and MEC6). Otherwise, the required resource in the cloud will be reserved in the graph (e12 edge between device 6 and CC6)

Step 1: Request classification

Initialize $G^t = G^e = \emptyset$

foreach $i \in \bar{N}$ do

if $T_i^{req} \leq \epsilon$ then
| add i to G^t

end

else
| add i to G^e

end

end

Step 2: Delay-aware graph creation

foreach $i \in G^t$ do

if $f_{mec} \geq \frac{Z_i}{T_i^{req} + T_i^{mec}}$ then

add vm_i to G^t ;

$f_{mec} \leftarrow f_{mec} - \frac{Z_i}{T_i^{req} + T_i^{mec}}$;

$E(i, vm_i) = \text{Equation 12}$;

end

foreach $j \in N \& j \notin G^t$ do

if $j = \text{neighbor}(i)$ then

$temp \leftarrow \text{Equation 9}$;

if $temp \leq T_i^{req}$ then

add j to G^t ;

$E(i, j) = temp$;

end

end

end

end

Step 3: Solve G^t by max matching with min cost [28];

$S^t \leftarrow \text{Selected nodes}$;

Algorithm 1: MEC offloading decision algorithm for delay sensitive devices

3.4 Complexity analysis

Proposed Algorithm 1 runs in three steps. Let N be the set of devices in the network and $\bar{N}$ be the set of requested devices. The first step is classifying the requested devices which runs

Step 1: Energy-efficient graph creation

```

foreach  $i \in G^e$  do
   $E_i^{mec} \leftarrow \text{Equation 13};$ 
  if  $E_i^{mec} < W_i^t$  then
    if  $f_{mec} > f_{min}$  then
      add  $vm_i$  to  $G^e$ ;
       $f_{mec} \leftarrow f_{mec} - f_{min};$ 
       $E(i, vm_i) = E_i^{mec};$ 
    end
  else
    add  $cloud_i$  to  $G^e$ ;
     $E(i, cloud_i) = E_i^{mec};$ 
  end
end
foreach  $j \in N$  &  $j \notin G^t$  &  $j \notin S^t$  do
  if  $j = \text{neighbor}(i)$  then
    add  $j$  to  $G^e$ ;
     $E(i, j) = \text{Equation 10};$ 
  end
end

```

Step 2: Solve G^e by max matching with min cost [28];

$S^e \leftarrow$ Selected nodes;

Algorithm 2: MEC offloading decision algorithm for energy sensitive devices

in $\mathcal{O}(|\bar{N}|)$. The second step is to create the delay-aware graph. In this step, the MEC nodes and corresponding edges are created in $\mathcal{O}(|\bar{N}|)$, and the D2D nodes and corresponding edges are created in $\mathcal{O}(|\bar{N}|(|N - \bar{N}|))$. As a result, the second step of the algorithm runs in $\mathcal{O}(|\bar{N}| + |\bar{N}|(|N - \bar{N}|))$. Now, the graph has been created with $|G^t|$ vertices and m edges which is solved by Edmond's Blossom maximum matching with minimum cost algorithm. A straightforward implementation of this algorithm runs in $\mathcal{O}(n^2m)$ for n number of vertices. A better running time $\mathcal{O}(m\sqrt{n}\log(n\mathcal{N}))$ is proposed in [29] which $\mathcal{N}$ bound the magnitude of any integer edge weight. However, the code has not been developed yet. Currently, the most efficient computational implementation has the complexity of $\mathcal{O}(nm \log n)$ that developed in [28] and is used in this study. Therefore, joining the complexity of all three steps yields the complexity of the proposed Algorithm 1 as:

$$\mathcal{O}(|\bar{N}| + |\bar{N}^t| + |\bar{N}^t|(|N - \bar{N}|) + |G^t|m \log |G^t|) \quad (16)$$

According to the fact that $N \geq \bar{N} \geq \bar{N}^t$, the computational complexity of EEDOS Algorithm 1 can be further explained as $\mathcal{O}(\text{Max}(cn, nm \log n))$ which is polynomial. Since the proposed Algorithm 2 is consisted of two steps of completing and solving energy-efficient graph (G^e), the run time is calculated similar to the 16 as the following which is also polynomial ($N \geq \bar{N} \geq \bar{N}^e$):

$$\mathcal{O}(|\bar{N}^e| + |\bar{N}^e|(|N - S^t|) + |G^e|m \log |G^e|) \quad (17)$$

4 Performance evaluation

To measure the performance of EEDOS and compare it with the existing studies, the following steps has been taken:

- Investigate the performance of EEDOS through simulation
- Simulate the D2D crowd system [19,20]
- Simulate the selective MEC offloading scheme [16]
- Validate the superior of EEDOS by comparing the results of the previous steps

The main reason of choosing these two kind of studies is that, both of them addressed task offloading problem in a same way and our goad is to integrate them. In [19,20], energy efficiency problem was formulated for D2D collaboration and in [16], delay optimization of task executing by MEC was formulated. To verify the effectiveness of the EDDOS scheme, a simulated environment is developed in SimuLTE [30]. The SINR of each D2D and cellular communication can be extracted from the log file of the simulation. D2D data rate between devices and cellular data rate of each device can be calculated form the SINR values using the Shannon Formula 4. The scenario considers two types of mobile phones with real energy consumption that were measured by [31]. The tasks were assigned randomly according to the parameters in 100 iterations to gain the average run-time and energy saving parameters. The maximum distance of each device to the BS were considered 100m. In accordance with the outcomes of the D2D and cellular links, the energy efficient and delay aware graphs were created.

For the task assignment, face recognition algorithm with 420KB of input image size that needs 1000 MCycles of computational resource [1] has been considered. Table 2 represents the simulation parameters. In the initialisation step, the simulation assigned a random load to devices, then tasks were created based on the variable frequencies (30% to 70% of devices in the network). Devices first calculated their capabilities, and if they found out that they could not meet the

Table 2 Simulation parameters

Parameter	Value
Edge CPU cycles	15 GHz
Cell size	100m
Maximum cellular transmit power	600mW
Maximum D2D transmit power	200mW
Maximum D2D bandwidth	20 MHz
Maximum D2D distance	200m
Device CPU cycles	[1.4,1.6] GHz
CPU load	[30–70]%

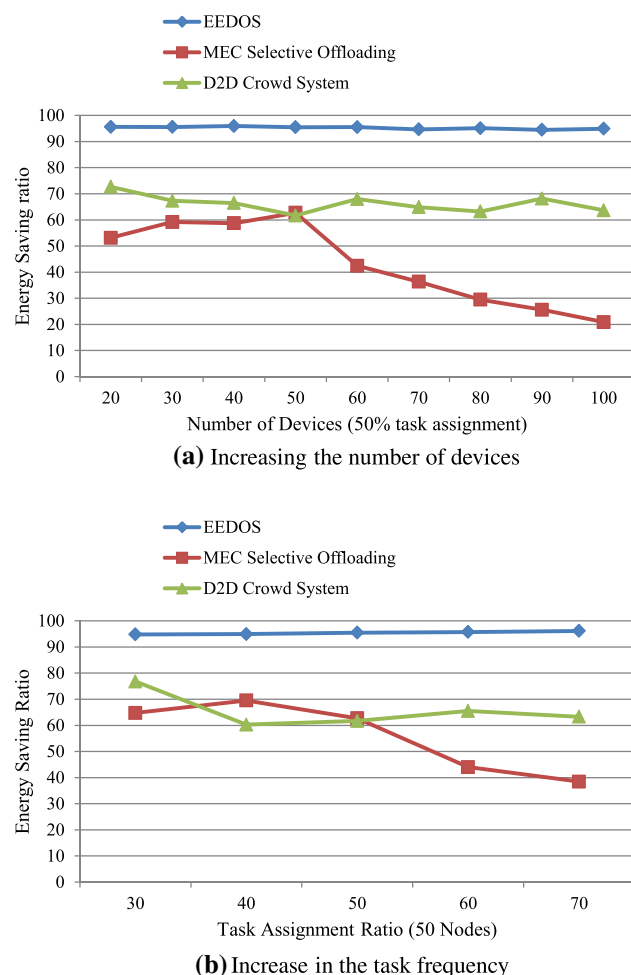


Fig. 3 Energy saving ratio

deadline, and/or they did not have sufficient energy, they sent an offloading request to the MEC.

4.1 Simulation results

Figure 3, depicts the energy saving ratio of EEDOS in comparison with two existing studies. In this figure, Y-axis is the percent of energy saving when the offloading scheme is compared with a case that each device executed the task on its local hardware. The X-axis in Fig. 3a is the number of devices in the simulation. In this case, the possibility of having a task is fixed to 50% to see the effect of increasing the number of devices in the network on the performance of EEDOS. The graph shows that EEDOS performed steady energy saving in comparison to MEC offloading. Although change margin for the D2D collaboration scenario is just 16%, but with the increase in the number of nodes, the energy saving of the edge-only scheme degrade dramatically to 21%. The reason is the overloading the edge server by increase in the number of offloading requests. Therefore, the edge server cannot

accept any other request, while in D2D-enabled schemes like EEDOS and [20], nearby idle devices handle the work in case of overloading the edge server.

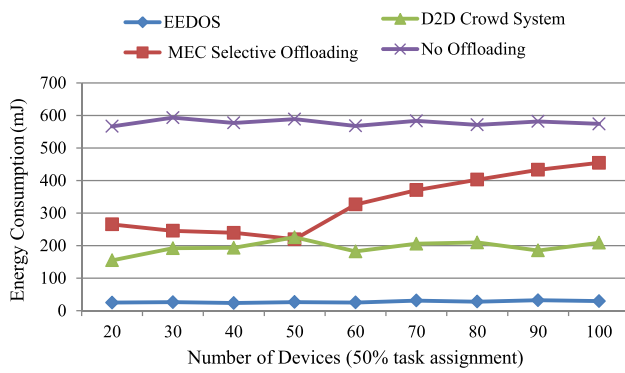
Similarly, Fig. 3b presents the energy saving ratio, but the number of devices are fixed in all the simulation iterations to see the effect of increasing the task assignment ratio on the performance of the proposed scheme. In this case, the possibility of having a task is variable from 30 to 70%.

There is a critical logical difference between sub-figures in Fig. 3. In the former figure, when the number of devices increased, the number of idle devices in the proximity of the devices that have a task will be increased as well. In the latter, when the task assignment ratio is increased, the number of idle devices in the proximity will be decreased. However, in both cases, EEDOS shows a steady energy saving from 94.8 to 95.9%. The reason is the integration of the capabilities of MEC and idle devices in the network. When the number of idle devices increase in the network, there are more resources in the mobile network for collaboration. In contrast, when the number of idle devices decrease, EEDOS can use the resources of MEC to run the offloaded tasks. As a result, EEDOS performed well in both cases.

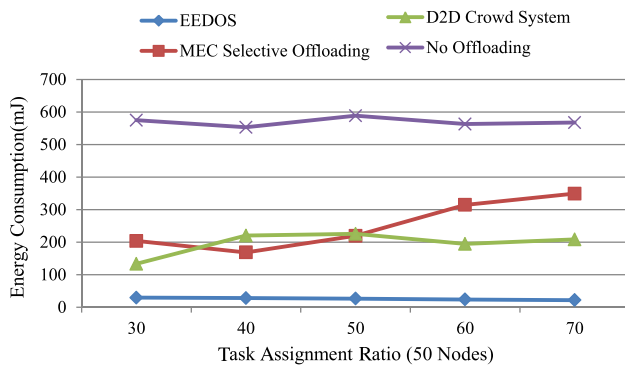
Authors in [20] present 50% of energy efficiency for D2D offloading when they compare running the task on local hardware. However, the current simulation achieves the average of 60% for their scheme. The reason for this difference is that in their scheme, if a user can perform its task, it will send its parameters to the edge server. Then the edge server informs the device that they can do it by themselves. In this study, it is considered that all of the devices that have a task do not have sufficient energy to run it. Therefore, they cannot complete the task, and they need to offload the task to MEC or a neighbor device. This approach is more common in the MEC offloading studies such as [16].

Average energy consumption to perform a task is shown in Fig. 4. The calculations are based on the real smartphone energy consumption measurement in [31].

The possibility of having a task is fixed in Fig. 4a, and the number of mobile devices is variable from 20 to 100 nodes. Four scenarios have been shown in the figure as: (a) No offloading: Means that devices run the task on their local hardware without getting help from the network, (b) MEC offloading: Based on the work presented in the [16], a device can run the task locally or offloads it to the MEC if offloaded execution consumes less energy, (c) D2D offloading: Based on the work in the [20], a device can run the task locally or if the MEC decides that it consumes less energy offloads it to a neighbor, and (d) EEDOS: A device can run the task locally, offloads to MEC, or a neighbor, if offloading consumes less energy. The figure shows that mobile devices in EEDOS consume as low as 21.8mJ to 29.60mJ. After EEDOS, D2D offloading scheme performs better with 133.3mJ to 225.3mJ, in comparison to the MEC



(a) Increasing the number of devices



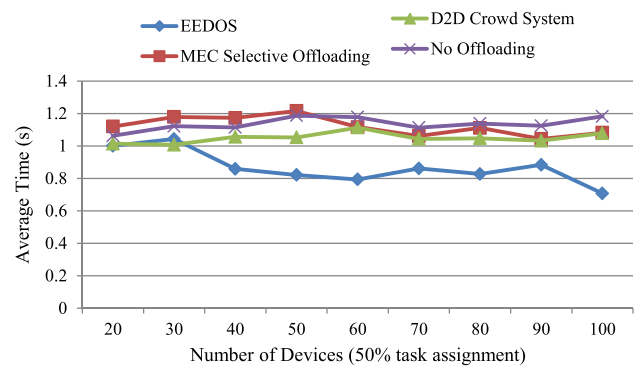
(b) Increase in the task frequency

Fig. 4 Energy Consumption

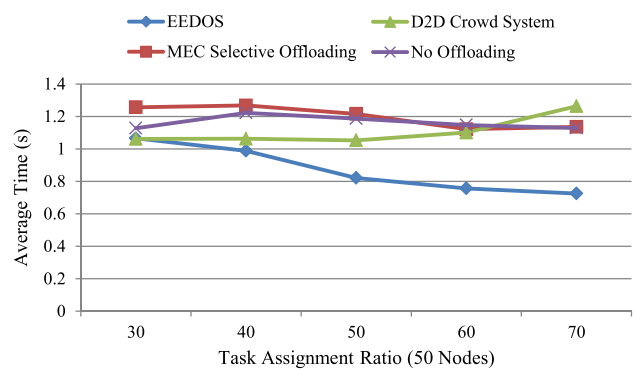
offloading scheme that mobile devices consume high energy from 168.5mJ to 349mJ. Again, as the number of devices increases the performance of the MEC offloading scheme degrades dramatically. It is because overloading the MEC with many number of offloading requests.

Figure 4b presents the same metric of energy consumption for the mobile devices, but with the increase in the task assignment ratio for the fixed number of devices in the network. It must be noted that running the task on local hardware as no offloading legend in both figures, consume considerably higher energy than all offloading schemes (552.9 mJ to 593.2 mJ).

Figure 5a demonstrates the average time needed to execute the task with the increase in the number of requesting devices. In this case, more idle nodes are participating in the collaboration. As a result, in EEDOS, the average execution time is reduced, as it can be seen when the number of devices is more than 30. In the simulation, it is supposed that idle nodes will dedicate all of their remaining CPU power to perform the task while edge server adjusts its resources based on the delay requirements. It means that for delay sensitive tasks, MEC collaboration takes precisely 1.0s, but for D2D offloading, it might be less if the offloading destination (idle node) has high computation capacity.



(a) Increasing number of devices



(b) Increase in the task frequency

Fig. 5 Execution delay

Figure 5b, presents the results for the fixed number of devices, and variable possibility of having a task from 30 to 70%, to see the effect of task assignment ratio on the energy consumption. It must be noted that, the no offloading results which devices run the task on their hardware is presented as a reference for the comparison. If a devices does not have sufficient energy to perform a task, obviously it cannot produce the output and contribute to the average execution time. However, it is calculated to show that beside energy saving in EEDOS, the execution time is improved significantly as well.

Figure 6, compares the number of successful task offloading in EEDOS and MEC offloading scheme that is presented in [16]. In the simulation of their work, the number of offloaded tasks is related to the computation capacity of the edge server. As a result, after 18 numbers of pre-assigning the resources for offloading, it is not possible to accept any request. In contrast, EEDOS, leverages the capability of idle nodes in the proximate of the requested device to increase the chance of successful offloading. In EEDOS, it is obvious that with the increase in the number of devices, the number of successful tasks are increased with the same rate. In contrast, in MEC offloading scheme, it can only perform 18 number of offloading at most.

Figure 7, enumerates the number of missed deadlines. This means that because of the failure in offloading and lack of

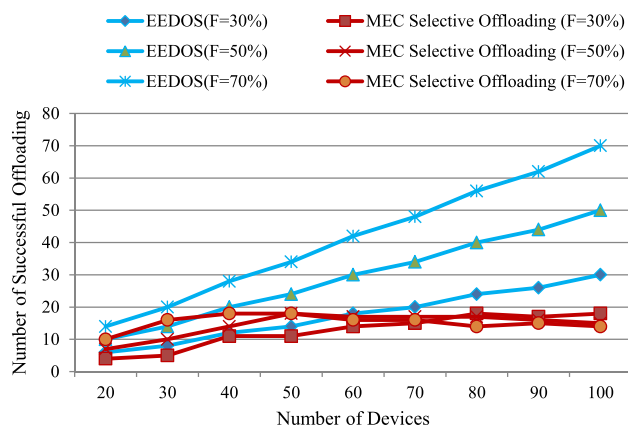


Fig. 6 Offloaded requests

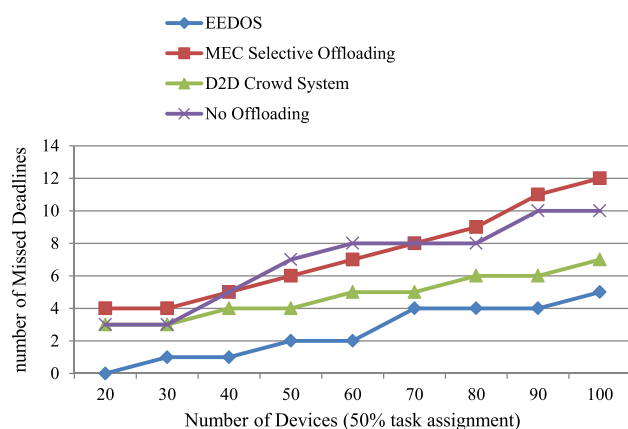


Fig. 7 Missed deadlines

enough computation power, some number of execution times exceed the deadline. As mentioned previously, no offloading values is presented to compare with offloading schemes. It is clear that EEDOS and D2D offloading scheme that leverage the computational capabilities of the massive number of devices in the network, perform better than local executing the tasks and MEC offloading.

One of the primary motivations of this work is to alleviate the workload on the edge computing server. Overloading the edge server is one of the main limitations of the MEC offloading schemes. By integrating D2D offloading and MEC offloading, EEDOS mitigates this problem. As depicted in Fig. 8, more than 61 GHz of computation capacity is required in the edge-only scheme which refers to the [16] study. This is observed with 100 number of devices that 70% of them have a task to offload. However, in EEDOS, D2D communication and collaboration decrease this amount by 71% to the value of 17.5 GHz. On the same time, when the network is not busy (20 devices with 30% task assignment), EEDOS needs only 2.72GHz of CPU, but MEC offloading requires 17.22GHz CPU resource in the edge server.

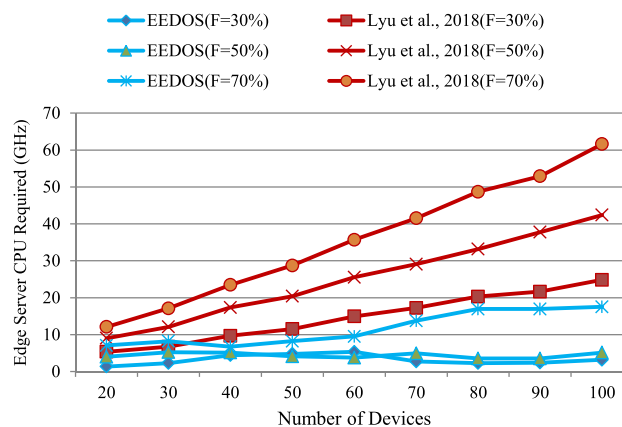


Fig. 8 Edge CPU required to handle all offloading requests and meet deadline

It can be observed from the Fig. 8 that MEC offloading “required edge server CPU”, is a linear function that has a direct relation with the number of devices. This means that with the increase in the number of devices, the amount of required CPU capacity is increased with the same rate. However, in EEDOS, the required edge server CPU is relatively same by the increase the number of devices. This is because the massive number of idle devices are available in the network to cooperate in the offloading scheme that decrease the load on the edge server.

5 Limitations of the study

In this study, we suppose that every idle device in the proximity is willing to collaborate on offloading, while it is not a realistic consideration. To motivate users for collaboration, there must be some incentives. This could be an interesting future research direction. Recently, social networks gain much attention to solve this issue in D2D-enabled networks [32,33]. However, this solution cannot be leveraged for IoT devices because of their nature. Additionally, IoT devices are more vulnerable against hackers because of their limited computation power. Recently, leveraging Blockchain technology in MEC for providing secure IoT environment is proposed by authors in [34]. Incorporating the idea of virtual currencies in [35] and Blockchain framework for IoT environment in [34] could be an excellent solution for this problem.

6 Conclusion

In this paper, an Energy-Efficient and Delay-aware Offloading Scheme (EEDOS) based on device to device collaboration in Mobile Edge Computing has been described. EEDOS

proposed to overcome the lack of integrated D2D and MEC framework as well as to address the energy and delay costs in a joint approach. EEDOS receives the task execution and offloading request on Mobile Edge Computing (MEC) server and classifies offloading requests based on the deadline and energy constraint. The proposed EEDOS is capable to find the proper offloading destination by utilizing the maximum matching with minimum cost graph algorithm. Through simulation, it has been shown that EEDOS achieved 95 of energy efficiency in comparison of no-offloading task execution and outperformed existing studies in term of energy efficiency with an improved delay in task execution.

Acknowledgements The study is supported by Project No.: BK067-2015 from University of Malaya, and the Fundamental Research Grant Scheme (FRGS), Project: FP007-2016 from Ministry of Higher Education, Malaysia.

Compliance with ethical standards

Conflict of interest On behalf of all authors, the corresponding author states that there is no conflict of interest.

References

1. Soyata, T., Muraleedharan, R., Funai, C., Kwon, M., & Heinzelman, W. (2012). Cloud-Vision: Real-time face recognition using a mobile-cloudlet-cloud acceleration architecture. In *Proceedings—IEEE symposium on computers and communications* (pp. 59–66).
2. Ali, F. A., Simoons, P., Verbelen, T., Demeester, P., & Dhoedt, B. (2016). Mobile device power models for energy efficient dynamic offloading at runtime. *Journal of Systems and Software*, 113, 173–187.
3. Akpakwu, G., Silva, B., Hancke, G., & Abu-Mahfouz, A. M. (2017). A survey on 5g networks for the internet of things: Communication technologies and challenges. *IEEE Access*, 6, 3619–3647.
4. Varma, A., Prabhakar, S., & Jayavel, K. (2017). Gas leakage detection and smart alerting and prediction using IoT. In *2nd international conference on computing and communication technologies (IC3T)* (pp. 327–333).
5. Hou, X., Li, Y., Chen, M., Wu, D., Jin, D., & Chen, S. (2016). Vehicular fog computing: A viewpoint of vehicles as the infrastructures. *IEEE Transactions on Vehicular Technology*, 65(6), 3860–3873.
6. Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7), 1645–1660.
7. Vallati, C., Virdis, A., Mingozzi, E., & Stea, G. (2015). Exploiting LTE D2D communications in M2M Fog platforms: Deployment and practical issues. In *IEEE world forum on internet of things, WF-IoT 2015 - proceedings* (pp. 585–590).
8. Dolui, K., & Datta, S. K. (2017). Comparison of edge computing implementations: Fog computing, cloudlet and mobile edge computing. In *GIoTS 2017—Global internet of things summit, proceedings*.
9. Chávez-Santiago, R., Szydelko, M., Kliks, A., Foukalas, F., Hadad, Y., Nolan, K. E., et al. (2015). 5G: The convergence of wireless communications. *Wireless Personal Communications*, 83(3), 1617–1642.
10. Reznik, A., Arora, R., Cannon, M., Cominardi, L., Featherstone, W., Frazao, R., Giust, F., Kekki, S., Li, A., Sabella, D., Turyagyenda, C., & Zheng, Z. (2017) ETSI white paper #20: Developing software for multi-access edge computing. In *ETSI* (20).
11. Jaffry, S., Hasan, S. F., Gui, X., & Kuo, Y. W. (2017). Distributed device discovery in ProSe environments. In *IEEE region 10 annual international conference, Proceedings/TENCON 2017-Decem* (pp. 614–618).
12. Feng, D., Lu, L., Yuan-Wu, Y., Li, G., & Li, S. (2014). Device-to-device communications in cellular networks. In *IEEE communications magazine* (pp 49–55), (April).
13. Alsahag, A. M., Ali, B. M., Noordin, N. K., et al. (2016). Maximum rate resource allocation algorithms with multiuser diversity and QoS support for downlink OFDMA based WiMAX system. *Telecommunication Systems*, 63(1), 1–14.
14. Bonomi, F., Milito, R., Zhu, J., & Addepalli, S. (2012). Fog computing and its role in the internet of things. In *Proceedings of the first edition of the MCC workshop on Mobile cloud computing* (pp. 13–16).
15. Zhang, K., Mao, Y., Leng, S., Zhao, Q., Li, L., Peng, X., et al. (2016). Energy-efficient offloading for mobile edge computing in 5G heterogeneous networks. *IEEE Access*, 4, 5896–5907.
16. Lyu, X., Tian, H., Jiang, L., Vinel, A., Maharjan, S., Gjessing, S., et al. (2018). Selective offloading in mobile edge computing for the green internet of things. *IEEE Network*, 32(1), 54–60.
17. Tran, T., & Pompili, D. (2019). Joint Task offloading and resource allocation for multi-server mobile-edge computing networks. *IEEE Transactions On Vehicular Technology*, 68(1), 856–868.
18. Pu, L., Chen, X., Xu, J., & Fu, X. (2016). D2D fogging: An energy-efficient and incentive-aware task offloading framework via network-assisted D2D collaboration. *IEEE Journal on Selected Areas in Communications*, 34(12), 3887–39014.
19. Chen, X., & Zhang, J. (2017). When D2D meets cloud: Hybrid mobile task offloadings in fog computing. In *IEEE international conference on communications* (pp. 0–5).
20. Chen, X., Pu, L., Gao, L., Wu, W., & Wu, D. (2017). Exploiting massive D2D collaboration for energy-efficient mobile edge computing. *IEEE Wireless Communications*, 24(4), 64–71.
21. Orakzai, F., Iqbal, M., Naeem, M., & Ahmad, A. (2018). Energy efficient joint radio resource management in D2D assisted cellular communication. *Telecommunication Systems*, 69(4), 505–517.
22. Ali, S., & Ahmad, A. (2017). Resource allocation, interference management, and mode selection in device-to-device communication: A survey. *Transactions On Emerging Telecommunications Technologies*, 28(7), e3148. <https://doi.org/10.1002/ett.3148>.
23. Diao, X., Zheng, J., Wu, Y., & Cai, Y. (2019). Joint computing resource, power, and channel allocations for D2D-assisted and NOMA-based mobile edge computing. *IEEE Access*, 7, 9243–9257.
24. Xing, H., Liu, L., Xu, J., & Nallanathan, A. (2019). Joint task assignment and resource allocation for D2D-enabled mobile-edge computing. *IEEE Transactions On Communications*, 1–1.
25. Yan, H., Zhang, X., Chen, H., Zhou, Y., Bao, W., & Yang, L. (2019). DEED: Dynamic energy-efficient data offloading for IoT applications under unstable channel conditions. *Future Generation Computer Systems*, 96, 425–437.
26. Salman, M. I., Mansoor, A. M., Jalab, H. A., Sabri, A. Q. M., & Ahmed, R. (2018). A joint evaluation of energy-efficient downlink scheduling and partial CQI feedback for LTE video transmission. *Wireless Personal Communications*, 98(1), 189–211.
27. Edmonds, J. (1965). Maximum matching and a polyhedron with 0,1-vertices. *Journal of Research of the National Bureau of Standards Section B, Mathematics and Mathematical Physics*, 69B(1 and 2), 125.
28. Kolmogorov, V. (2008). Blossom V : A new implementation of a minimum cost perfect matching algorithm 1 Introduction 2

Background: Edmonds' blossom algorithm. *Mathematical Programming Computation*, 1, 1–14.

29. Duan, R., Pettie, S., & Su, H. (2018). Scaling algorithms for weighted matching in general graphs. *ACM Transactions On Algorithms*, 14(1), 1–35.
30. Virdis, A., Stea, G., & Nardini, G. (2015). Simulating LTE/LTE-advanced networks with simuLTE. *Advances in Intelligent Systems and Computing*, 402, 83–105.
31. Kwak, J., Kim, Y., Lee, J., & Chong, S. (2015). DREAM: Dynamic resource and task allocation for energy minimization in mobile cloud systems. *IEEE Journal on Selected Areas in Communications*, 33(12), 2510–2523.
32. Yi, C., Huang, S., & Cai, J. (2018). An incentive mechanism integrating joint power, channel and link management for social-aware D2D content sharing and proactive caching. *IEEE Transactions on Mobile Computing*, 17(4), 789–802.
33. Huynh, D. T., Wang, X., Duong, T. Q., Vo, N. S., & Chen, M. (2018). Social-aware energy efficiency optimization for device-to-device communications in 5G networks. *Computer Communications*, 120, 102–111.
34. Li, R., Song, T., Mei, B., Li, H., Cheng, X., & Sun, L. (2018). Blockchain for large-scale internet of things data storage and protection. *IEEE Transactions on Services Computing*. <https://doi.org/10.1109/tsc.2018.2853167>.
35. Buttyan, L., & Hubaux, J.p. (2001). Nuglets: A virtual currency to stimulate cooperation in self-organized mobile Ad Hoc networks. *Technical report DSC* (pp. 1–15).

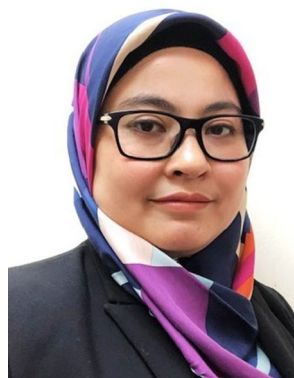
Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Ramtin Ranji received the Master of Computer Science in 2019 from the University of Malaya, Malaysia. He is a multidisciplinary wireless engineer with eight years of industrial experience. From 2017 to 2019, he was Research Assistant at the University of Malaya in the Software Engineering and CompChem labs. His research interests include edge computing, wireless communication, software defined networking (SDN) and Blockchain.



Ali Mohammed Mansoor received the B.Sc. degree from Amman University, Jordan, and his MSc from University Putra Malaysia, both in computer Science and distributed computing in 2005 and 2008, respectively. He received his PhD degree in Communications and Network Engineering from University Putra Malaysia in 2014. Currently he is a senior lecturer at the Faculty of Computer Science and Information Technology in University of Malaya, Malaysia. Previously, he was a lecturer at the Department of Computer Engineering, Faculty of engineering, Aden University, Yemen. He also worked on broadband networks in MIMOS Berhad, Malaysia. His main research interests are wireless communication and networking, resource management and QoS, Internet of Things (IoT), LTE Advance networks, 5G emerging wireless technologies and edge computing.



Asmiza Abdul Sani is a Senior Lecturer at the Department of Software Engineering, Faculty of Computer Science and Information Technology, University of Malaya. Her recent work focuses on IoT frameworks and her research interest includes formal software specifications and model-driven engineering. She is also currently looking into Blockchain modelling and specifications.